

Comparing a Search Algorithm and an Evolutionary Algorithm on the 0/1 Knapsack Problem

1. Problem Context and Formal Definition

The optimisation problem selected for this report is the **0/1 knapsack problem**, a standard combinatorial optimisation benchmark in which each item must be either fully selected or fully rejected. The goal is to choose a subset of items that maximises total value without exceeding a fixed capacity. The key constraint is therefore feasibility under a hard weight limit, while the decision variables are binary inclusion choices for each item. This problem is a strong benchmark for comparing exact and heuristic methods because it is easy to define, has a clear optimality criterion, and still produces non-trivial search behaviour as instance structure changes. Recent work also shows that knapsack remains an active testbed for exact, heuristic, and hybrid optimisation methods rather than only a classical textbook example (Bringmann, 2024; Burashnikov, 2024; Zhou et al., 2023). In mathematical terms, let there be n items. Item i has weight w_i and value v_i and the knapsack has capacity C . The binary decision variable is $x_i \in \{0,1\}$, where $x_i = 1$ means the item is selected and $x_i = 0$ means it is excluded. The objective is to maximise $\sum_{i=1}^n v_i x_i$ subject to $\sum_{i=1}^n w_i x_i \leq C$.

A feasible solution is any binary vector whose total selected weight does not exceed capacity. Because each item is either taken or skipped, the search space contains 2^n possible subsets. That combinatorial growth makes the problem suitable for comparing a method that proves optimality through pruning with a method that searches stochastically for high-quality feasible solutions.

Table 1. Formal problem definition and evaluation requirements.

Aspect	Definition in this report	Why it matters
Decision variables	Binary vector $x = (x_1, \dots, x_n)$ where each $x_i \in \{0,1\}$	Matches both tree-search branching and chromosome encoding
Objective	Maximise total item value	Provides the common quality metric for both algorithms
Constraint	Total selected weight must not exceed capacity C	Creates the core trade-off between benefit and limited resource
Feasible solution	Any binary vector satisfying the weight limit	Prevents invalid overweight solutions from being accepted
Search difficulty	Combinatorial space of 2^n subsets	Explains why exact and heuristic methods may behave differently
Evaluation objective	Compare runtime, solution quality, and reliability	Makes clear that the study evaluates speed, accuracy, and consistency together

This report compares best-first Branch and Bound and a Genetic Algorithm on three explicit dimensions: speed (runtime), solution quality (optimality gap and optimum-hit performance), and reliability (whether the method consistently returns exact or near-exact solutions under the same budget). That evaluation objective is important because exact and heuristic methods should not be judged by one metric alone.

2. Problem Definition and Algorithm Selection

Once the problem is defined as a binary constrained-selection task, the algorithm choice can be justified more strategically. The exact method should exploit the structure of partial include/exclude decisions and prune aggressively when a branch cannot beat the incumbent solution. The evolutionary method should work naturally with binary encodings, tolerate large search spaces, and maintain feasible solutions under a hard capacity constraint.

2.1 Best-first Branch and Bound as the search algorithm

The exact algorithm selected is best-first Branch and Bound (B&B). This is a strong exact choice for the 0/1 knapsack problem because each partial assignment naturally corresponds to a node in a binary decision tree, and the problem admits a tight, interpretable upper bound via the fractional

knapsack relaxation. At level k , the algorithm has already made inclusion or exclusion decisions for the first k sorted items and branches on item $k + 1$.

The key strength of B&B is its bounding mechanism. After sorting items by decreasing value-to-weight ratio, each partial solution is relaxed to a fractional knapsack problem. The resulting upper bound estimates the best value still reachable from that node. If that bound is not better than the best feasible solution already found, the node is pruned. This is exactly why B&B is a strategic choice here: it keeps the exactness guarantee while still exploiting problem-specific structure to avoid exploring the full tree.

A priority queue expands the node with the highest upper bound first. This strategy quickly identifies strong incumbent solutions and improves subsequent pruning while preserving optimality guarantees.

2.2 Genetic Algorithm as the evolutionary algorithm

The evolutionary algorithm selected is a Genetic Algorithm (GA) with binary encoding, tournament selection, one-point crossover, bit-flip mutation, elitism, and feasibility repair. This is a suitable choice because the problem already has a natural binary representation, the search space becomes very large even at moderate n , and solution value provides a direct fitness measure.

GA was selected over greedy search, simulated annealing, and tabu search because it naturally supports binary chromosome representations and population-based exploration. Unlike greedy methods, it can recover from poor early decisions through crossover and mutation. Compared with single-solution approaches such as simulated annealing and tabu search, GA maintains multiple candidate solutions simultaneously, improving exploration of large combinatorial search spaces.

Feasibility is maintained through a repair operator. If crossover or mutation produces an overweight solution, selected items with the lowest value-to-weight ratios are removed until the capacity constraint is satisfied. Remaining capacity is then filled greedily using the best unselected ratios. This ensures that all evaluated chromosomes remain feasible without requiring penalty parameters.

Recent studies also continue to apply Genetic Algorithms successfully to knapsack variants, often combining evolutionary search with exact optimisation components to improve solution quality and scalability (Adouani, 2025).

2.3 Why this is a meaningful pair to compare

The comparison is meaningful for more than the simple label of “one exact, one heuristic.” Both methods operate on the same binary decision structure, both are adapted to the specific geometry of the knapsack problem, and both use value-to-weight information in problem-specific ways. B&B uses it to compute an admissible upper bound for pruning; the GA uses it inside the repair operator to restore feasibility and improve offspring quality. This makes the comparison fair because both algorithms are structurally matched to the problem rather than one being heavily customised and the other generic.

Table 2. Why the two algorithms are meaningfully different.

Comparison dimension	Best-first Branch and Bound	Genetic Algorithm
Search style	Deterministic exact search over a state-space tree	Stochastic population-based search over binary chromosomes
Guarantee	Can prove optimality if it completes	Cannot prove optimality
Problem-specific mechanism	Fractional-knapsack upper bound for pruning	Binary encoding plus feasibility repair
Main advantage	Exactness and certificate of optimality	Good feasible solutions under predictable runtime
Main weakness	Runtime can rise sharply when bounds weaken	Results depend on stochastic search and parameter settings
Why it is a fair comparator	Uses the same binary decision structure and problem-specific ratio information	Uses the same binary decision structure and problem-specific ratio information

This pairing therefore tests a genuine methodological trade-off: exactness versus budgeted search predictability, not simply “old algorithm versus modern algorithm.” Recent comparative and survey work on knapsack and related combinatorial optimisation problems supports this distinction between exact, heuristic, and hybrid search strategies (Ghadi et al., 2023; Rashed et al., 2024; Souto et al., 2024; Zhou et al., 2023). More recent work continues to demonstrate the relevance of both exact Branch-and-Bound methods and evolutionary approaches for knapsack-type optimisation problems, particularly when balancing solution quality against computational cost (Adouani, 2025; Encz et al., 2025).

3. Methodology and Implementation

This section explains how the two methods were implemented and evaluated on the 0/1 knapsack problem. Both operated on the same instances and solution representation so that performance differences could be attributed to search strategy rather than modelling inconsistency.

3.1 Overall evaluation framework

The study uses a common evaluation pipeline. For each generated instance, a dynamic programming oracle first computes the true optimum. Branch and Bound and the Genetic Algorithm are then run on the same instance, and their outputs are compared in terms of runtime, optimality gap, and reliability. Using the same oracle baseline strengthens the comparison because both algorithms are judged against the same ground truth.

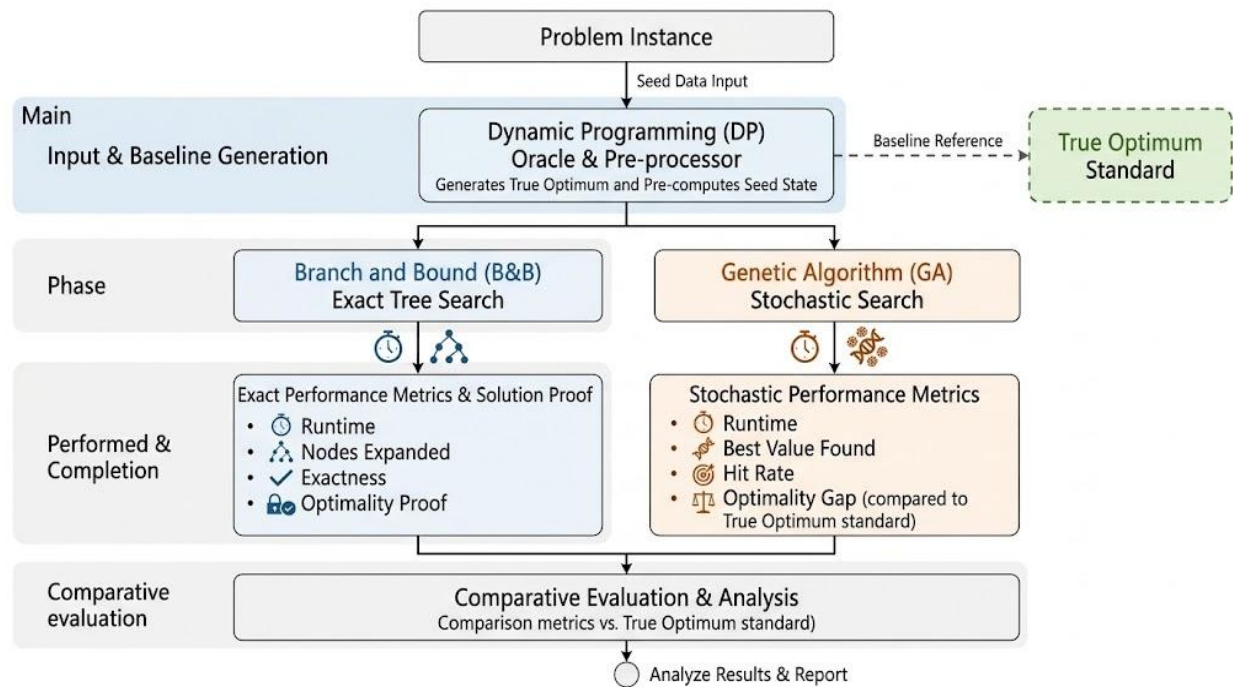


Figure 1. Workflow used to evaluate the two optimisation methods on the 0/1 knapsack problem. Both the exact search method and the evolutionary method are assessed against the same dynamic programming optimum.

Table 3. Experimental configuration.

Setting	Value used	Purpose
Instance families	Uncorrelated; strongly correlated	Tests behaviour under structurally different bounding conditions
Problem sizes	50, 100, 150, 200 items	Tracks performance as problem size increases in regular steps
Instances per setting	3	Reduces dependence on one atypical instance
GA runs per instance	5	Estimates stochastic reliability rather than reporting one run
Branch and Bound limit	5 seconds per instance	Imposes a practical wall-clock budget on exact search
GA parameters	Population 100, generations 180, crossover 0.9, mutation $1/n$, elite size 2	Balances exploration, exploitation, and runtime stability

3.2 Problem representation

Each 0/1 knapsack instance is defined by item weights, item values, and a knapsack capacity. A candidate solution is represented as a binary vector $x \in \{0, 1\}^n$, where $x_i = 1$ indicates that item i is selected and $x_i = 0$ indicates that it is excluded. Using the same binary representation for both algorithms is important for methodological consistency. It also makes the problem especially suitable for the selected methods, because B&B naturally branches on binary inclusion decisions, while the GA directly evolves binary chromosomes.

3.3 Implementation Details

Both algorithms were implemented against the same binary representation and evaluated against the dynamic programming oracle, but their internal mechanics differ substantially. Additional implementation details are provided in Appendix A.

For Branch and Bound, items are first sorted in descending order of value-to-weight ratio. Each node stores the current depth, accumulated weight, accumulated value, and an upper bound. The upper bound is computed using the fractional-knapsack relaxation: after taking all remaining whole items that still fit, the algorithm adds the value of the next item fractionally for the residual

capacity. If a node at level ℓ has current value V and current weight W , its upper bound is $V + \sum_{j=\ell}^{k-1} v_j + (C - W - \sum_{j=\ell}^{k-1} w_j) \cdot \frac{v_k}{w_k}$, where items from ℓ onward are considered in sorted ratio order, all items up to $k - 1$ fit completely, and item k is the first one that fits only fractionally. Because this is an admissible upper bound, any node whose bound is not greater than the incumbent best value is pruned. A priority queue implements best-first expansion, so the node with the largest bound is always expanded next.

For the Genetic Algorithm, the initial population is generated randomly and then immediately repaired to feasibility. Parent selection uses tournament selection of size three, where the fittest individual among three randomly sampled candidates becomes the parent. One-point crossover is applied with probability 0.9. Mutation flips each bit independently with probability $1/n$, which keeps the expected number of mutations per offspring close to one.

The repair operator restores feasibility whenever an offspring exceeds the capacity limit. Selected items are removed in ascending value-to-weight ratio order until feasibility is restored, after which remaining capacity is greedily filled using the best unselected ratios that still fit. This avoids the need for penalty coefficients while maintaining feasible solutions throughout the search.

B&B terminates when the priority queue is exhausted or the 5-second wall-clock limit is reached. The GA terminates after 180 generations. Fixed stopping rules were used to ensure consistent runtime comparisons across experiments.

3.4 Experimental design and fairness of comparison

The experimental design was chosen to test both scalability and structural sensitivity. Uncorrelated instances produce a wider spread of value-to-weight ratios, which generally strengthens the fractional upper bound and pruning in Branch and Bound. Strongly correlated instances create similar ratios and weaken the bound, making them a useful stress case for exact search.

Problem sizes of 50, 100, 150, and 200 items were used to show scaling behaviour while remaining practical for repeated experimentation. Five GA runs per instance were used to assess stochastic reliability, and a 5-second cap was imposed on Branch and Bound as a practical compute budget.

4. Results and Discussion

The results show that relative performance depends strongly on instance structure. On uncorrelated instances, B&B was exact and extremely fast. On strongly correlated instances, its pruning power weakened and the GA became more competitive in both runtime predictability and average solution quality under the time limit.

4.1 Performance on uncorrelated instances

Table 4. Mean performance on uncorrelated instances.

<i>n</i>	BB runtime (s)	BB gap (%)	BB exact rate (%)	GA runtime (s)	GA hit-optimum rate (%)
50	0.0002	0.0	100.0	0.7194	100.0
100	0.0006	0.0	100.0	1.2976	93.33
150	0.0011	0.0	100.0	1.9041	73.33
200	0.0023	0.0	100.0	2.5194	46.67

On uncorrelated instances, B&B achieved a 0.0% mean gap and a 100% exact success rate across all sizes while remaining extremely fast. The GA also produced near-optimal solutions, but its optimum-hit rate declined as problem size increased, showing that exact optimum recovery became less reliable as the search space expanded.

4.2 Performance on strongly correlated instances

Table 5. Mean performance on strongly correlated instances.

<i>n</i>	BB runtime (s)	BB gap (%)	BB exact rate (%)	GA runtime (s)	GA hit-optimum rate (%)
50	0.5342	0.0	100.0	0.6975	100.0
100	1.9856	0.2424	66.67	1.2882	100.0
150	3.3432	0.3822	33.33	1.8913	73.33
200	2.9680	0.1789	66.67	2.5506	40.0

Strongly correlated instances reduced the effectiveness of Branch and Bound because similar value-to-weight ratios weakened the upper bound and increased node expansions. Under the 5-second limit, exact success declined, and small optimality gaps appeared. The GA maintained relatively stable runtime and continued to produce high-quality solutions, indicating lower sensitivity to this change in instance structure.

4.3 Reliability and Predictability

The main trade-off is between exactness and runtime predictability. Branch and Bound can certify optimality, but its runtime depends heavily on pruning effectiveness. The Genetic Algorithm cannot guarantee optimality, yet its runtime is more stable because population size and generation count are fixed. B&B is therefore preferable when exact solutions are essential, whereas the GA is attractive when near-optimal solutions are sufficient under limited compute budgets.

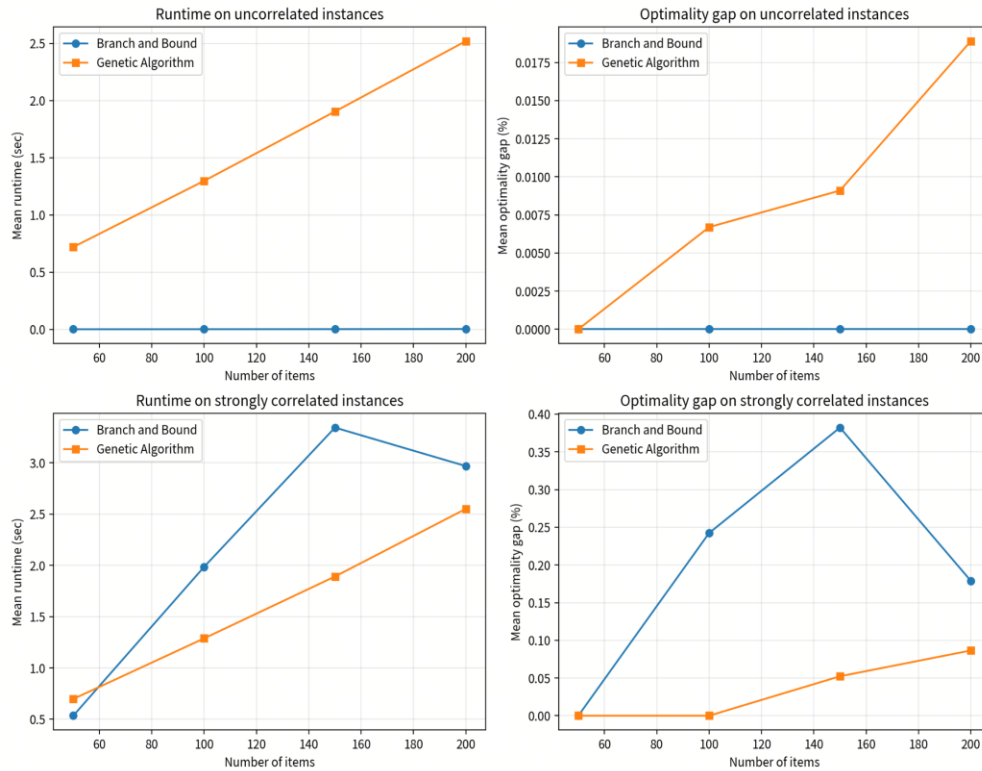


Figure 2. Mean runtime and optimality-gap comparison for Branch and Bound and the Genetic Algorithm on uncorrelated and strongly correlated instance families.

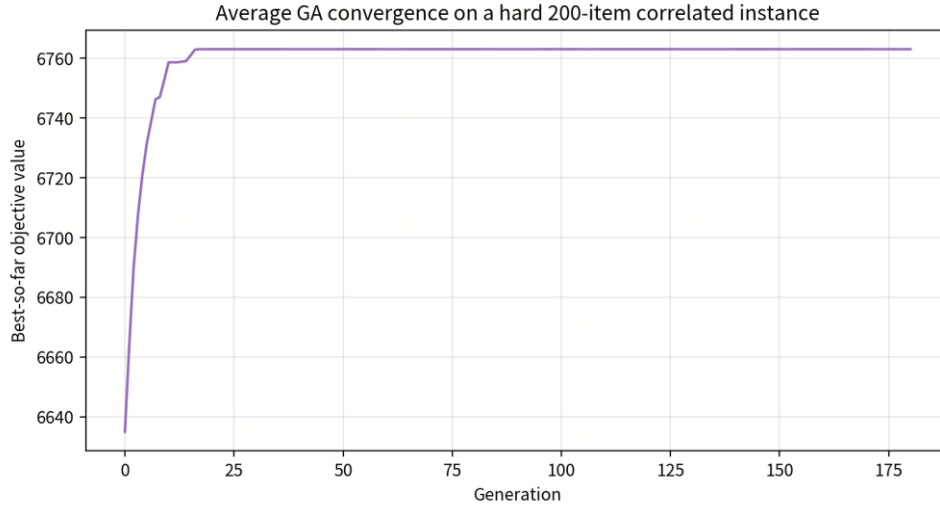


Figure 3. Average best-so-far objective value over generations for one difficult 200-item strongly correlated instance.

The results therefore support a more analytical conclusion than a simple winner-loser judgment. If the requirement is certified optimality, B&B is the stronger method whenever the upper bound remains informative. If the requirement is stable runtime with high-quality feasible solutions, the GA becomes more attractive, especially on structurally difficult instances where exact pruning weakens.

4.4 Limitations

The findings should be interpreted with several limitations in mind. First, the experiment uses only two benchmark families and three instances per size-family setting, so it cannot represent the full diversity of knapsack structures. Second, the GA was repeated only five times per instance, which is enough to reveal variability but not enough for a strong statistical characterisation of stochastic performance. Third, the 5-second limit is a useful practical budget, but different budgets could shift the relative advantage between the algorithms. These limitations do not invalidate the comparison, but they do constrain how broadly the results should be generalised.

5. Conclusion

Neither algorithm is uniformly superior. On uncorrelated instances, best-first Branch and Bound was clearly stronger because effective pruning delivered certified optima almost instantly. On strongly correlated instances, pruning weakened and the Genetic Algorithm became more competitive through more stable runtime and strong solution quality. The choice between the two therefore depends on whether exact proof or predictable budgeted performance matters more.

References

- Adouani, Y. (2025). *A genetic algorithm combined with mathematical programming to solve Generalised Quadratic Multiple Knapsack Problem*. *International Journal of Mathematics in Operational Research*, 32(2), 199–213.
- Bringmann, K. (2024). *Knapsack with small items in near-quadratic time* (arXiv:2308.03075v3). arXiv.
<https://arxiv.org/abs/2308.03075>
- Burashnikov, E. (2024). Branch-and-Bound and Dynamic Programming Approaches for the Knapsack Problem. *Operations Research Forum*, 5, 98. <https://doi.org/10.1007/s43069-024-00372-2>
- Encz, K. I., Mastrolilli, M., & Vercesi, E. (2025). *Branch-and-Bound Algorithms as Polynomial-time Approximation Schemes*. *Proceedings of ICALP 2025 / arXiv*.
- Ghadi, Y. Y., AlShoul, T., Nezami, Z. I., Ali, H., Asif, M., Aljuaid, H., & Ahmad, S. (2023). An efficient optimizer for the 0/1 knapsack problem using group counseling. *Computational Intelligence and Neuroscience*, 2023, Article 10280447. <https://doi.org/10.1155/2023/10280447>
- Rashed, N. A., Ali, Y. H., & Rashid, T. A. (2024). Advancements in optimization: Critical analysis of evolutionary, swarm, and behavior-based algorithms. *Algorithms*, 17(9), 416. <https://doi.org/10.3390/a17090416>
- Souto, G., Miceli, C., Simonetti, L., & González, P. H. (2024). Machine learning assisted hybrid genetic algorithm applied to the knapsack problem with forfeits. In *Proceedings of the 41st International Conference on Machine Learning: LatinX in AI Research Workshop* (PMLR Vol. 235). *Journal of LatinX in AI Research*.
<https://doi.org/10.52591/lxai202407275>
- Zhou, Y., Shi, Y., Wei, Y., Luo, Q., & Tang, Z. (2023). Nature-inspired algorithms for 0-1 knapsack problem: A survey. *Neurocomputing*, 554, 126630. <https://doi.org/10.1016/j.neucom.2023.126630>

Appendix

Appendix A: Implementation Mechanics and Complexity Analysis

To effectively contrast exact and evolutionary optimisation, the study evaluates three distinct computational procedures: a baseline oracle, a deterministic search, and a stochastic heuristic.

The true optimum for each instance is first computed using a dynamic programming oracle. Implemented with a one-dimensional table, this baseline operates with a time complexity of $O(nC)$ and a space complexity of $O(C)$, where n represents the number of items and C represents the knapsack capacity. This provides a definitive benchmark for evaluating the search algorithms.

The exact search method is implemented as a best-first Branch and Bound algorithm. Candidate items are sorted in descending order by their value-to-weight ratio, enabling the computation of a highly informative upper bound via the fractional knapsack relaxation. The search tree tracks the depth, accumulated weight, accumulated value, and bound value at each node. Utilizing a priority queue, the algorithm expands nodes with the most promising upper bounds first, branching on binary inclusion decisions. The search space is heavily pruned: branches that violate the capacity constraint are immediately discarded, as are nodes whose upper bounds cannot surpass the incumbent best feasible solution. While this method guarantees absolute optimality when run to completion, its worst-case time complexity is $O(2^n)$. Because practical performance is tightly coupled to the discriminatory power of the upper bound, a time cap is introduced to assess the algorithm's robustness under limited computational budgets.

The evolutionary solver is structured as a classical binary Genetic Algorithm (GA), chosen for its simplicity, interpretability, and direct alignment with the problem's binary state space. Operating on the same binary encoding as the exact search, the GA initializes randomly and evolves through tournament selection, one-point crossover, bit-flip mutation (applied at a rate of $1/n$), and elitism to preserve top-performing individuals. Crucially, rather than relying on penalty functions, the algorithm enforces feasibility via a dedicated repair operator. This operator strips items with the lowest value-to-weight ratios from overweight chromosomes until the capacity constraint is met, subsequently backfilling any residual capacity with promising items using a greedy heuristic. A key advantage of this evolutionary approach is its structural predictability: the GA operates with an approximate time complexity of $O(P \times G \times n)$, where P is the population size and G is the generation count.

This methodological contrast is central to the evaluation framework. Branch and Bound delivers certified exactness but risks computational explosion on challenging combinatorial instances, whereas the Genetic Algorithm sacrifices definitive optimality for a stable, scalable, and highly predictable search effort.

Appendix B: Complete Python Implementation (knapsack_compare.py)

```
import csv
import heapq
import math
import random
import statistics
import time

from dataclasses import dataclass
from pathlib import Path

import matplotlib.pyplot as plt

OUTPUT_DIR = Path('output')
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

@dataclass
class Item:
    index: int
    weight: int
    value: int
    ratio: float

class KnapsackInstance:
    def __init__(self, weights, values, capacity, kind, seed):
        self.weights = list(weights)
        self.values = list(values)
        self.capacity = int(capacity)
        self.kind = kind
        self.seed = seed
        self.n = len(weights)

def generate_instance(n, kind, seed):
    rng = random.Random(seed)
    weights = [rng.randint(10, 100) for _ in range(n)]
    if kind == 'uncorrelated':
        values = [rng.randint(10, 100) for _ in range(n)]
    elif kind == 'strongly_correlated':
        values = [w + 10 for w in weights]
    else:
        raise ValueError(f'Unknown kind: {kind}')
    capacity = int(sum(weights) * 0.5)
    return KnapsackInstance(weights, values, capacity, kind, seed)
```

```

def dynamic_programming_optimum(instance):
    capacity = instance.capacity
    dp = [0] * (capacity + 1)
    for w, v in zip(instance.weights, instance.values):
        for c in range(capacity, w - 1, -1):
            candidate = dp[c - w] + v
            if candidate > dp[c]:
                dp[c] = candidate
    return max(dp)

def branch_and_bound(instance, time_limit=10.0):
    items = [
        Item(i, instance.weights[i], instance.values[i], instance.values[i] /
instance.weights[i])
        for i in range(instance.n)
    ]
    items.sort(key=lambda x: x.ratio, reverse=True)
    start = time.perf_counter()

    def bound(level, weight, value):
        if weight >= instance.capacity:
            return value
        total_weight = weight
        total_value = value
        j = level
        while j < instance.n and total_weight + items[j].weight <= instance.capacity:
            total_weight += items[j].weight
            total_value += items[j].value
            j += 1
        if j < instance.n:
            remaining = instance.capacity - total_weight
            total_value += remaining * items[j].ratio
        return total_value

    best_value = 0
    best_taken_sorted = [0] * instance.n
    nodes_expanded = 0
    root_bound = bound(0, 0, 0)
    heap = [(-root_bound, 0, 0, 0, tuple())]

    while heap:
        if time.perf_counter() - start > time_limit:
            break
        neg_bound, level, weight, value, decisions = heapq.heappop(heap)

```

```

        upper = -neg_bound
        if upper <= best_value:
            continue
        if level == instance.n:
            continue
        nodes_expanded += 1
        item = items[level]

        new_weight = weight + item.weight
        new_value = value + item.value
        if new_weight <= instance.capacity:
            take_decisions = decisions + (1,)
            if new_value > best_value:
                best_value = new_value
                best_taken_sorted = list(take_decisions) + [0] * (instance.n -
len(take_decisions))
                take_bound = bound(level + 1, new_weight, new_value)
                if take_bound > best_value:
                    heapq.heappush(heap, (-take_bound, level + 1, new_weight, new_value,
take_decisions))

            skip_decisions = decisions + (0,)
            skip_bound = bound(level + 1, weight, value)
            if skip_bound > best_value:
                heapq.heappush(heap, (-skip_bound, level + 1, weight, value,
skip_decisions))

        elapsed = time.perf_counter() - start
        optimality_proven = len(heap) == 0

        sorted_to_original = [0] * instance.n
        for sorted_pos, item in enumerate(items):
            sorted_to_original[item.index] = best_taken_sorted[sorted_pos]

    return {
        'best_value': best_value,
        'solution': sorted_to_original,
        'runtime_sec': elapsed,
        'nodes_expanded': nodes_expanded,
        'optimality_proven': optimality_proven,
    }

def repair_solution(bits, instance):
    bits = bits[:]
    total_weight = sum(b * w for b, w in zip(bits, instance.weights))
    if total_weight > instance.capacity:

```



```

        selected = [i for i, b in enumerate(bits) if b == 1]
        selected.sort(key=lambda i: instance.values[i] / instance.weights[i])
        for i in selected:
            if total_weight <= instance.capacity:
                break
            bits[i] = 0
            total_weight -= instance.weights[i]
        unselected = [i for i, b in enumerate(bits) if b == 0]
        unselected.sort(key=lambda i: instance.values[i] / instance.weights[i],
reverse=True)
        for i in unselected:
            if total_weight + instance.weights[i] <= instance.capacity:
                bits[i] = 1
                total_weight += instance.weights[i]
        return bits

def solution_value(bits, instance):
    weight = sum(b * w for b, w in zip(bits, instance.weights))
    value = sum(b * v for b, v in zip(bits, instance.values))
    return weight, value

def genetic_algorithm(instance, seed, population_size=100, generations=180,
crossover_rate=0.9, mutation_rate=None, elite_size=2):
    rng = random.Random(seed)
    n = instance.n
    if mutation_rate is None:
        mutation_rate = 1.0 / n

    def random_individual():
        bits = [1 if rng.random() < 0.5 else 0 for _ in range(n)]
        return repair_solution(bits, instance)

    def fitness(individual):
        return solution_value(individual, instance)[1]

    def tournament(population, k=3):
        contestants = rng.sample(population, k)
        return max(contestants, key=fitness)

    def crossover(parent1, parent2):
        if rng.random() >= crossover_rate:
            return parent1[:], parent2[:]
        point = rng.randint(1, n - 1)
        child1 = parent1[:point] + parent2[point:]
        child2 = parent2[:point] + parent1[point:]

```

```

        return repair_solution(child1, instance), repair_solution(child2, instance)

def mutate(individual):
    child = individual[:]
    for i in range(n):
        if rng.random() < mutation_rate:
            child[i] = 1 - child[i]
    return repair_solution(child, instance)

start = time.perf_counter()
population = [random_individual() for _ in range(population_size)]
best = max(population, key=fitness)
best_value = fitness(best)
history = [best_value]

for _ in range(generations):
    ranked = sorted(population, key=fitness, reverse=True)
    new_population = [ind[:] for ind in ranked[:elite_size]]
    while len(new_population) < population_size:
        p1 = tournament(ranked)
        p2 = tournament(ranked)
        c1, c2 = crossover(p1, p2)
        c1 = mutate(c1)
        c2 = mutate(c2)
        new_population.append(c1)
        if len(new_population) < population_size:
            new_population.append(c2)
    population = new_population
    current = max(population, key=fitness)
    current_value = fitness(current)
    if current_value > best_value:
        best = current[:]
        best_value = current_value
    history.append(best_value)

elapsed = time.perf_counter() - start
return {
    'best_value': best_value,
    'solution': best,
    'runtime_sec': elapsed,
    'history': history,
}

def summarise(values):
    return statistics.mean(values), (statistics.pstdev(values) if len(values) > 1 else
0.0)

```

```

def run_experiments():
    sizes = [50, 100, 150, 200]
    kinds = ['uncorrelated', 'strongly_correlated']
    instances_per_setting = 3
    ga_runs_per_instance = 5
    bb_time_limit = 5.0

    summary_rows = []
    detailed_rows = []
    convergence_records = []

    for kind in kinds:
        for n in sizes:
            optimum_values = []
            bb_runtimes = []
            bb_nodes = []
            bb_gaps = []
            bb_success = 0

            ga_runtimes = []
            ga_gaps = []
            ga_success = 0
            ga_values_all = []

            for instance_id in range(instances_per_setting):
                seed = 1000 + (0 if kind == 'uncorrelated' else 5000) + n * 10 +
instance_id

                instance = generate_instance(n, kind, seed)
                optimum = dynamic_programming_optimum(instance)
                optimum_values.append(optimum)

                bb_result = branch_and_bound(instance, time_limit=bb_time_limit)
                bb_gap = 100.0 * (optimum - bb_result['best_value']) / optimum
                bb_runtimes.append(bb_result['runtime_sec'])
                bb_nodes.append(bb_result['nodes_expanded'])
                bb_gaps.append(bb_gap)
                if bb_result['best_value'] == optimum and
bb_result['optimality_proven']:
                    bb_success += 1

                instance_ga_values = []
                histories = []
                for ga_run in range(ga_runs_per_instance):
                    ga_seed = seed * 100 + ga_run
                    ga_result = genetic_algorithm(instance, seed=ga_seed)

```

```

        gap = 100.0 * (optimum - ga_result['best_value']) / optimum
        ga_runtimes.append(ga_result['runtime_sec'])
        ga_gaps.append(gap)
        ga_values_all.append(ga_result['best_value'])
        instance_ga_values.append(ga_result['best_value'])
        histories.append(ga_result['history'])
        if ga_result['best_value'] == optimum:
            ga_success += 1

    avg_history = [statistics.mean(step_values) for step_values in
zip(*histories)]

    if n == 200 and kind == 'strongly_correlated' and instance_id == 0:
        convergence_records = avg_history

    detailed_rows.append({
        'kind': kind,
        'n': n,
        'instance_id': instance_id,
        'optimum': optimum,
        'bb_best_value': bb_result['best_value'],
        'bb_runtime_sec': round(bb_result['runtime_sec'], 6),
        'bb_nodes_expanded': bb_result['nodes_expanded'],
        'bb_gap_pct': round(bb_gap, 4),
        'bb_optimality_proven': bb_result['optimality_proven'],
        'ga_mean_value': round(statistics.mean(instance_ga_values), 4),
        'ga_best_value': max(instance_ga_values),
        'ga_mean_gap_pct': round(100.0 * (optimum -
statistics.mean(instance_ga_values)) / optimum, 4),
        'ga_best_gap_pct': round(100.0 * (optimum -
max(instance_ga_values)) / optimum, 4),
    })

    bb_runtime_mean, bb_runtime_std = summarise(bb_runtimes)
    bb_gap_mean, bb_gap_std = summarise(bb_gaps)
    bb_nodes_mean, bb_nodes_std = summarise(bb_nodes)
    ga_runtime_mean, ga_runtime_std = summarise(ga_runtimes)
    ga_gap_mean, ga_gap_std = summarise(ga_gaps)

    summary_rows.append({
        'kind': kind,
        'n': n,
        'instances': instances_per_setting,
        'ga_runs_total': instances_per_setting * ga_runs_per_instance,
        'optimum_mean': round(statistics.mean(optimum_values), 2),
        'bb_runtime_mean_sec': round(bb_runtime_mean, 4),
        'bb_runtime_std_sec': round(bb_runtime_std, 4),
        'bb_nodes_mean': round(bb_nodes_mean, 2),

```

```

        'bb_nodes_std': round(bb_nodes_std, 2),
        'bb_gap_mean_pct': round(bb_gap_mean, 4),
        'bb_gap_std_pct': round(bb_gap_std, 4),
        'bb_exact_success_rate_pct': round(100.0 * bb_success /
instances_per_setting, 2),
        'ga_runtime_mean_sec': round(ga_runtime_mean, 4),
        'ga_runtime_std_sec': round(ga_runtime_std, 4),
        'ga_gap_mean_pct': round(ga_gap_mean, 4),
        'ga_gap_std_pct': round(ga_gap_std, 4),
        'ga_hit_optimum_rate_pct': round(100.0 * ga_success /
(instances_per_setting * ga_runs_per_instance), 2),
        'ga_mean_value': round(statistics.mean(ga_values_all), 2),
    })

summary_path = OUTPUT_DIR / 'knapsack_summary.csv'
detailed_path = OUTPUT_DIR / 'knapsack_detailed.csv'

with summary_path.open('w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=list(summary_rows[0].keys()))
    writer.writeheader()
    writer.writerows(summary_rows)

with detailed_path.open('w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=list(detailed_rows[0].keys()))
    writer.writeheader()
    writer.writerows(detailed_rows)

plot_summary(summary_rows)
if convergence_records:
    plot_convergence(convergence_records)

return summary_rows, detailed_rows

def plot_summary(summary_rows):
    kinds = ['uncorrelated', 'strongly_correlated']
    fig, axes = plt.subplots(2, 2, figsize=(12, 9))

    for idx, kind in enumerate(kinds):
        rows = [r for r in summary_rows if r['kind'] == kind]
        sizes = [r['n'] for r in rows]
        bb_runtime = [r['bb_runtime_mean_sec'] for r in rows]
        ga_runtime = [r['ga_runtime_mean_sec'] for r in rows]
        bb_gap = [r['bb_gap_mean_pct'] for r in rows]
        ga_gap = [r['ga_gap_mean_pct'] for r in rows]

        ax1 = axes[idx][0]

```

```

ax1.plot(sizes, bb_runtime, marker='o', label='Branch and Bound')
ax1.plot(sizes, ga_runtime, marker='s', label='Genetic Algorithm')
ax1.set_title(f'Runtime on {kind.replace("_", " ")} instances')
ax1.set_xlabel('Number of items')
ax1.set_ylabel('Mean runtime (sec)')
ax1.grid(True, alpha=0.3)
ax1.legend()

ax2 = axes[idx][1]
ax2.plot(sizes, bb_gap, marker='o', label='Branch and Bound')
ax2.plot(sizes, ga_gap, marker='s', label='Genetic Algorithm')
ax2.set_title(f'Optimality gap on {kind.replace("_", " ")} instances')
ax2.set_xlabel('Number of items')
ax2.set_ylabel('Mean optimality gap (%)')
ax2.grid(True, alpha=0.3)
ax2.legend()

fig.tight_layout()
fig.savefig(OUTPUT_DIR / 'knapsack_performance.png', dpi=200)
plt.close(fig)

def plot_convergence(avg_history):
    fig, ax = plt.subplots(figsize=(8, 4.5))
    ax.plot(range(len(avg_history)), avg_history, color='tab:purple')
    ax.set_title('Average GA convergence on a hard 200-item correlated instance')
    ax.set_xlabel('Generation')
    ax.set_ylabel('Best-so-far objective value')
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(OUTPUT_DIR / 'ga_convergence.png', dpi=200)
    plt.close(fig)

if __name__ == '__main__':
    summary_rows, detailed_rows = run_experiments()
    print('Summary rows:')
    for row in summary_rows:
        print(row)
    print(f'Wrote {len(summary_rows)} summary rows and {len(detailed_rows)} detailed
rows.')

```